

Hoja de Trabajo 2

Reinforcement Learning

Francis Aguilar, 22243

José Marchena, 22398

Task 1 (Entrega Parcial)

Responda las siguientes preguntas de forma clara, concisa y argumentada. No basta con definir, se espera que usted conecte el concepto con sus implicaciones prácticas o estratégicas.

1. Elijan un sistema real que pueda modelarse como un k -armed bandit. Su modelado debe incluir:

Motor de recomendación de ejercicios en una app de aprendizaje de idiomas (tipo Duolingo) En cada sesión el agente debe decidir qué ejercicio mostrarle a un usuario ahora mismo, sin conocer de antemano si ese ejercicio lo va a enganchar, aburrir o frustrar.

1.1 Definición formal de los k brazos: qué representa cada acción y por qué el conjunto de acciones es finito y discreto en este dominio.

Cada brazo $a \in \{1, \dots, k\}$ puede representar una combinación de (tema, tipo de ejercicio, nivel de dificultad) que el sistema puede ofrecer en el siguiente turno. Por ejemplo:

a	Tema	Tipo de pregunta	Dificultad
1	Vocabulario	Opción múltiple	Fácil
2	Gramática (subjuntivo)	Completar espacio	Difícil
3	Comprensión auditiva	Completar espacio	Fácil
4	Traducción libre	Completar espacio	Difícil
5	Pronunciación	Hablar en el micrófono	Nivel intermedio

El conjunto es finito y discreto porque cada opción está organizada en unidades y tipos de ejercicio predefinidos. Por tanto cada combinación es una decisión cerrada, donde el agente elige entre un menú finito de "Próximo ejercicio".

1.2 Diseño justificado de la función de recompensa: qué se mide, cómo se mide, y qué supuestos distribucionales son razonables para ese dominio. Si la recompensa no es gaussiana, argumentenlo.

Qué se mide: una señal compuesta de engagement + desempeño, por ejemplo:

$R_t = w_1 \cdot (\text{completó el ejercicio}) + w_2 \cdot (\text{tiempo antes de la próxima sesión})$ (conociendo el que se busca maximizar completación de ejercicios, y minimizar el tiempo de próxima sesión)

Cómo se mide: vía eventos de la app (clic en "completar", apertura de la app al día siguiente), agregados por usuario y por brazo.

Supuestos distribucionales: La recompensa combina dos componentes de naturaleza distinta, y ninguna es gaussiana:

- El término de **completó el ejercicio** es una variable Bernoulli (0 o 1), acotada y discreta.
- El término de **tiempo antes de la próxima sesión** es una variable de duración (tiempo hasta que el usuario regresa), que es no negativa por definición y típicamente muestra una distribución con cola larga a la derecha (la mayoría de los usuarios regresan rápido, unos pocos tardan mucho o nunca regresan). Se podría aplicar normalización para mantener este valor en un rango manejable como 0 y 1, lo que nos podría dar una distribución como Tanh.
- Al combinar ambos términos en R_t , el resultado es una **mezcla de una variable binaria con una variable de cola larga**, es decir, una distribución bimodal y asimétrica, muy alejada de la campana simétrica que asume un modelo gaussiano. Por eso, para estimar $q^*(a)$ se recurre a promedios empíricos (muestrales o con paso constante), sin asumir una forma paramétrica gaussiana de fondo.

1.3 Análisis de estacionariedad: ¿los valores verdaderos $q^*(a)$ cambian con el tiempo en este sistema? ¿En qué escala temporal? ¿Qué implicaciones tiene eso para el algoritmo de actualización?

Los valores verdaderos $q^*(a)$ **no son estacionarios**, y cambian en al menos tres escalas temporales distintas:

- **Escala de minutos (dentro de una sesión):** el usuario se cansa o se frustra conforme avanza la sesión, por lo que su tolerancia a la dificultad y su probabilidad de completar un ejercicio cambian en tiempo real.
- **Escala de días/semanas:** el usuario **aprende**. Un brazo que antes generaba baja recompensa (porque el usuario fallaba y tardaba en volver) puede genuinamente mejorar su $q^*(a)$ conforme el usuario domina el tema. Este es un cambio real, no ruido, y es justamente la señal que el sistema busca aprovechar.
- **Escala de meses:** cambios de motivación general (rachas, vacaciones, cambio de objetivo de aprendizaje) modifican qué tipo de ejercicio mantiene enganchado al usuario.

Implicación para el algoritmo de actualización: un promedio muestral clásico (paso $1/n$) trataría la mejora real del usuario como si fuera ruido estadístico y diluiría la señal reciente entre todo el historial acumulado, retrasando la detección de que el usuario ya domina un brazo antes difícil. Esto justifica usar una **regla de actualización con paso constante α** , que pondera más las observaciones recientes y permite que $Q_t(a)$ "siga" la curva de aprendizaje del usuario en vez de promediar todo su historial por igual.

1.4 Identificación de restricciones de exploración: ¿hay costos económicos, de seguridad o regulatorios que limiten cuánto puede explorar el sistema?

- **Costo de frustración y abandono (churn):** explorar repetidamente con ejercicios demasiado difíciles puede hacer que el usuario cierre la app y no regrese — una recompensa muy negativa y potencialmente irreversible, ya que a diferencia de otros dominios, aquí se puede perder al usuario por completo, no solo una interacción puntual.
- **Costo de aburrimiento:** explorar solo con ejercicios fáciles para "jugar seguro" tampoco es válido, ya que reduce la satisfacción y limita el progreso real del usuario (su nota o nivel).
- **Restricción pedagógica (zona de desarrollo próximo):** no cualquier exploración es aceptable — conviene explorar cerca del nivel actual del usuario y no de forma uniforme sobre todo el espacio de dificultad, ya que un salto de dificultad demasiado grande no aporta información útil y sí aumenta el riesgo de frustración.
- **Restricción de tiempo de sesión:** las sesiones son cortas (minutos al día), por lo que hay pocas oportunidades diarias de "probar" un brazo con cada usuario — el presupuesto de exploración es limitado y cada tirada exploratoria tiene un costo de oportunidad alto.

2. Con base en el análisis anterior, propongan y justifiquen:

2.1 La estrategia de selección de acción más apropiada para su dominio, con el valor concreto del hiperparámetro relevante (ϵ o c) y la justificación de ese valor.

Se propone **UCB (Upper Confidence Bound)** con $c = 1.5$, en lugar de ϵ -greedy.

La razón es que, dado el costo de frustración y abandono identificado en 1.4, no conviene explorar de forma aleatoria-uniforme como hace ϵ -greedy (que podría, con probabilidad ϵ , mostrarle a un usuario avanzado un ejercicio trivial o a un principiante uno absurdamente difícil, sin importar cuánta información ya se tiene sobre ese brazo). UCB en cambio explora de forma sistemática y decreciente: prioriza brazos con poca información ($N_t(a)$ bajo) y reduce la exploración sobre brazos que ya han sido probados muchas veces, concentrando el riesgo exploratorio donde realmente aporta información nueva.

Fórmula: $A_t = \operatorname{argmax}_a [Q_t(a) + c \cdot \sqrt{(\ln t / N_t(a))}]$

Se elige $c = 1.5$ como valor moderado: suficiente para cubrir el cold start con ~5 brazos disponibles en las primeras sesiones, pero no tan alto como para insistir en ejercicios difíciles ya fallidos varias veces solo por falta de datos, lo cual —dado el riesgo de churn— sería costoso. Valores típicos para bandits con recompensa acotada en $[0,1]$ rondan $c \in [1, 2]$; 1.5 es un punto medio conservador.

2.2 La regla de actualización de estimaciones (paso variable o constante) con justificación ligada al análisis de estacionariedad.

Dado que en 1.3 se estableció que el entorno **no es estacionario** (el usuario aprende y cambia su motivación con el tiempo), se usa una regla de **paso constante**:

$$Q_{n+1}(a) = Q_n(a) + \alpha \cdot (R_n - Q_n(a)), \text{ con } \alpha = 0.15$$

Se elige un α relativamente alto (comparado con un promedio muestral clásico) porque el número de observaciones por usuario y por brazo es limitado (pocas repeticiones de cada tipo de ejercicio por semana), por lo que conviene que $Q_t(a)$ reaccione rápido a cada nueva observación y capture la curva de aprendizaje del usuario sin esperar muchas repeticiones para "confiar" en el cambio.

2.3 Un ejemplo numérico con al menos seis observaciones que ilustre cómo evoluciona $Q_t(a)$ para dos acciones distintas bajo la regla elegida, mostrando cómo la estimación converge o se adapta.

Comparamos dos brazos para un mismo usuario: **A1 (Gramática — difícil)** y **A2 (Vocabulario — fácil)**, con $Q_0 = 0.5$ (valor neutro inicial) y $\alpha = 0.15$. La recompensa $R_t \in [0,1]$ combina completó el ejercicio y qué tan rápido regresó a la siguiente sesión, en 6 sesiones sucesivas:

Acción A1 (Gramática difícil)

t	R _t	Q _t antes	$Q_{t+1} = Q_t + 0.15(R_t - Q_t)$
1	0.2	0.500	0.455
2	0.3	0.455	0.432
3	0.6	0.432	0.457
4	0.7	0.457	0.494
5	0.8	0.494	0.539
6	0.9	0.539	0.594

Acción A2 (Vocabulario fácil)

t	R _t	Q _t antes	$Q_{t+1} = Q_t + 0.15(R_t - Q_t)$
1	0.9	0.500	0.560
2	0.8	0.560	0.596
3	0.7	0.596	0.622
4	0.5	0.622	0.604
5	0.4	0.604	0.573
6	0.3	0.573	0.532

Lectura del resultado:

En **A1**, Q_t parte bajo (el usuario falla y tarda en regresar al principio) pero crece consistentemente de 0.455 a 0.594: refleja que el usuario aprende con la práctica, y el paso constante permite que la estimación siga esa mejora real sin diluirla entre los primeros intentos fallidos.

En **A2**, Q_t parte alto pero decrece de 0.596 a 0.532: el ejercicio deja de generar el mismo engagement porque el usuario ya lo domina y empieza a aburrirse — justo el riesgo de "mantener los ejercicios muy simples" que se buscaba evitar.

Hacia $t=6$, ambos valores convergen en un rango similar (0.53–0.59). En ese punto, UCB —al tener probablemente menos observaciones acumuladas sobre A1 por haber sido explorado con más cautela— seguiría asignándole cierto peso exploratorio adicional, empujando gradualmente al usuario hacia contenido más desafiante sin abandonar el refuerzo de vocabulario por completo. Esto es exactamente el comportamiento buscado: subir la dificultad progresivamente sin generar frustración excesiva ni estancarse en lo trivial.

Task 2 (Entrega Final)

Realice y responda:

1. Implementen en Python un simulador de k-armed bandit con las tres estrategias vistas en clase: greedy puro, epsilon-greedy y UCB1. Su implementación debe incluir:
 - A. Una clase `KArmedBandit` con soporte para modo estacionario y no estacionario. En modo no estacionario, los valores $q^*(a)$ deben perturbarse cada cierto número de pasos configurable.
 - B. Una clase por cada estrategia con métodos `select_action` y `update`, implementando la regla incremental con soporte para paso variable y paso constante.
 - C. Un loop de experimento que corra al menos 500 episodios independientes de 1000 pasos, calculando recompensa promedio por paso y regret acumulado.
 - D. Cuatro gráficas: recompensa promedio y regret acumulado en el entorno estacionario, y las mismas dos métricas en el entorno no estacionario.
2. Con base en los resultados de la implementación:
 - A. Analicen el comportamiento del regret acumulado de cada estrategia en el entorno estacionario. ¿La jerarquía entre estrategias coincide con lo esperado teóricamente? ¿A partir de qué paso el regret de UCB comienza a diferenciarse del de epsilon-greedy?
 - B. Analicen qué ocurre en el entorno no estacionario. ¿Qué estrategia se adapta mejor? ¿Cómo cambia el resultado al usar paso constante versus paso variable?
 - C. Realicen un análisis de sensibilidad: corran epsilon-greedy con al menos tres valores distintos de ϵ y UCB con al menos tres valores de c . Grafiquen el regret final en función del hiperparámetro y discutan cuál rango de valores es más robusto.
 - D. Cierren con una sección de diseño aplicado: tomando los resultados anteriores, ¿qué configuración específica recomendarían para el sistema real que modelaron en la entrega parcial? Justifiquen la recomendación con evidencia de sus experimentos, no solo con argumentos teóricos.

